

Comprehensive Backend Implementation Plan

What I found from the repo

- **Stack already decided:** FastAPI + PostgreSQL 16 (asyncpg) + Redis 7 + MinIO + Logto OIDC — confirmed by docker-compose files
- **No backend directory yet** — starting from scratch
- **54 API endpoints** across 8 modules, all with typed contracts extracted from the frontend
- **Every domain has a state machine** (lease, payment, inspection, maintenance, onboarding, notification)
- **Notification channels:** WhatsApp (Meta Cloud API), Email (SendGrid), SMS (Twilio) — but see suggestion #1 below

Technology Choices (with reasoning)

Layer	Choice	Why
Framework	FastAPI 0.115	Async-native, auto OpenAPI, Pydantic v2
ORM	SQLAlchemy 2.0 (async) + asyncpg	Matches postgresql+asyncpg:// in docker-compose
Migrations	Alembic	Standard, supports autogenerate
Validation	Pydantic v2	Already in FastAPI, best Python schema library
Auth	python-jose + httpx (JWKS fetch)	Verifies Logto JWT without coupling to Logto SDK
Task Queue	Celery 5 + redis	Rent reminders, late fees, notification delivery
File Storage	boto3 (S3-compatible) → MinIO	Docker-compose already has MinIO
PDF	WeasyPrint	HTML→PDF, easy templating with Jinja2
Email	sendgrid-python	Already in .env.example
SMS	Twilio (swap to Africa's Talking later)	Already in .env.example
WhatsApp	httpx → Meta Cloud API	WHATSAPP_API_KEY already in .env
Logging	structlog + JSON	Structured output for log aggregation
Rate limiting	slowapi (ASGI middleware)	Prevents abuse per landlord/tenant
Testing	pytest-asyncio + httpx AsyncClient	Async test support

Suggested Improvements

- 1. Africa's Talking over Twilio for Uganda** — AT has direct MTN/Airtel connections, much lower latency and cost in East Africa. I'll design the SMS adapter so it's swappable with one config change.
- 2. MTN MoMo + Airtel Money payment callbacks** — The mock data shows method: "mobile_money_mtn". I'll add webhook receiver endpoints for both payment gateways so real payments can auto-reconcile. This is critical for Uganda.
- 3. Idempotency keys on all write operations** — Especially payments. Prevents duplicate charges when mobile networks drop mid-request.
- 4. Separate Celery worker service** — The docker-compose doesn't have it yet. I'll add it to docker-compose.local.yml.
- 5. Row-Level Security in Postgres** — Each landlord can only ever see their own data. Rather than filtering in every query, I'll set `app.current_landlord_id` as a Postgres session variable and add RLS policies at the DB level. Belt-and-suspenders on top of app-layer auth.
- 6. pgcrypto for signature data** — Lease signatures store `signatureDataUrl` (base64 PNG, potentially 50-200KB). Store these encrypted at rest using `pgp_sym_encrypt`.
- 7. GDPR anonymisation pipeline** — The `POST /tenants/{id}/anonymise` endpoint needs to zero out PII fields deterministically. I'll implement an Anonymiser class that runs in a transaction and emits an `GDPR_ERASURE_COMPLETED` event.
- 8. Webhook-first notifications** — Don't call external APIs synchronously in request path. Every notification goes through Celery → retry logic → delivery status webhook back.
- 9. Audit log as event sourcing (append-only)** — Lease audit is required. I'll extend this pattern to payments and onboarding so you have a full history for compliance.
- 10. OpenAPI tag grouping + Swagger auth** — Auto-generate docs with try-it-out working via bearer token.

Project Structure

```
backend/
├── app/
│   ├── api/v1/
│   │   ├── deps.py          # get_current_user, get_db, require_role
│   │   ├── properties.py
│   │   ├── units.py
│   │   ├── tenants.py
│   │   ├── leases.py
│   │   ├── payments.py
│   │   └── inspections.py
```

- | | └─ maintenance.py
- | | └─ notifications.py
- | | └─ analytics.py
- | | └─ upload.py
- | | └─ users.py
- | └─ core/
 - | | └─ config.py # pydantic-settings, all env vars typed
 - | | └─ security.py # JWKS fetch + JWT decode + role extraction
 - | | └─ middleware.py # request-id, CORS, rate limit
 - | | └─ logging.py # structlog JSON setup
 - | | └─ exceptions.py # AppError hierarchy + 422/404/403 handlers
- | └─ db/
 - | | └─ base.py # Base model with id, created_at, updated_at, deleted_at
 - | | └─ session.py # AsyncSessionLocal, get_db()
- | └─ events/
 - | | └─ bus.py # Redis pub/sub emit()
 - | | └─ handlers.py # on_lease_activated → generate rent schedules, etc.
- | └─ integrations/
 - | | └─ storage.py # S3Client (MinIO) — presign, upload, delete
 - | | └─ documents.py # PDF generation (WeasyPrint + Jinja2 templates)
 - | | └─ notifications/
 - | | | └─ base.py # NotificationProvider ABC
 - | | | └─ email.py # SendGrid
 - | | | └─ sms.py # Twilio / Africa's Talking (swappable)
 - | | | └─ whatsapp.py # Meta Cloud API
- | └─ modules/ # One sub-package per domain
- | └─ properties/
 - | | | └─ models.py # Property, Unit SQLAlchemy models
 - | | | └─ schemas.py # Pydantic request/response
 - | | | └─ service.py # Business logic (owns transactions)
 - | | | └─ repository.py # Pure DB queries, no business logic

- | | └─ tenants/ # + anonymiser.py, invite.py
- | | └─ leases/ # + pdf.py, signing.py
- | | └─ payments/ # + late_fees.py, schedules.py, export.py
- | | └─ inspections/
- | | └─ notifications/ # + templates.py, dispatcher.py
- | | └─ users/
- | └─ tasks/
- | | └─ celery_app.py
- | | └─ rent_reminders.py # Daily: find due rents, queue SMS/WhatsApp
- | | └─ late_fees.py # Daily: apply late fees after grace period
- | | └─ notification_delivery.py # Async send with retry
- | └─ utils/
- | | └─ pagination.py # Reusable paginate() helper
- | | └─ filters.py # QueryParams → SQLAlchemy WHERE clauses
- | | └─ gdpr.py # Anonymiser
- | | └─ reference.py # Generate KOL-L0001-style references
- | └─ workflows/
- | | └─ base.py # StateMachine ABC
- | | └─ lease.py
- | | └─ onboarding.py
- | | └─ payment.py
- | | └─ inspection.py
- | | └─ maintenance.py
- | | └─ notification.py
- | └─ main.py # FastAPI app factory, middleware, lifespan
- └─ alembic/
- | └─ env.py
- | └─ versions/
- └─ templates/ # Jinja2 HTML for PDF rendering
- | └─ lease_agreement.html
- └─ tests/

- | └─ conftest.py # Async test client, test DB
- | └─ test_properties.py
- | └─ test_leases.py
- | └─ test_payments.py
- └─ Dockerfile
- └─ Dockerfile.dev
- └─ pyproject.toml
- └─ .env.example

Database Schema (key design decisions)

Every table gets: id UUID PK, created_at, updated_at, deleted_at (soft delete), landlord_id FK (multi-tenancy).

users → id, email, name, role, logto_sub (unique), status

properties → + address JSONB, rules JSONB, amenities[], tags[], images[]

units → + rules JSONB (nullable, inherits from property)

tenants → + pii_encrypted BYTEA (NIN etc.), gdpr_consent_at, anonymised_at

tenant_invites → + token, expires_at, status

tenant_documents → + type, url, verified, expires_at

leases → + terms JSONB, clauses JSONB, signatures JSONB, state

lease_audit_log → append-only, no soft delete, leaseId + event + from/to state

rent_schedules → + period_start, period_end, due_date, state

payments → + category, method, state, idempotency_key UNIQUE

late_fees → + rent_schedule_id, fee_type, calculated_amount, state

deposits → + deductions JSONB, status

inspections → + checklist JSONB, photo_urls[], state

maintenance → + category, priority, state

notifications → + channel, trigger, state, retry_count

notification_templates → + trigger, channel, body, variables[]

Implementation Order (8 sprints)

Sprint	Scope	Why first
1	Scaffolding: project layout, config, DB session, base models, Alembic, Docker, health check	Everything depends on this
2	Auth: Logto JWKS verification, get_current_user dep, RBAC decorator, RLS session var	Blocks all other endpoints
3	Properties + Units CRUD + rules	Simplest domain, no state machine
4	Tenants: CRUD + invite flow + onboarding state machine + documents + GDPR anonymise	Core entity
5	Leases: full lifecycle + state machine + signatures + PDF generation + audit log	Depends on tenants + properties
6	Payments: CRUD + rent schedules + late fees + deposit + ledger + CSV/PDF export + Celery tasks	Depends on leases
7	Inspections + Maintenance, Analytics queries	Depends on properties + tenants
8	Notifications: templates + dispatcher + Celery delivery + channel adapters (email/SMS/WhatsApp)	Depends on all

Security & GDPR Checklist

-  All endpoints require valid JWT — no anonymous access except onboarding token flow
-  Role checks: landlord/manager can write; tenant read-only (their own data)
-  landlord_id filter on every query — never leak cross-landlord data
-  RLS policies in Postgres as second layer
-  Signature data encrypted at rest (pgcrypto)
-  GDPR erasure: anonymise() zeroes name, email, phone, NIN, replaces with REDACTED-{hash}
-  gdpr_consent_at required before creating a tenant profile
-  data_retention_until enforced by scheduled Celery task (auto-anonymise after period)
-  Rate limiting: 100 req/min per IP, 1000 req/min per authenticated user

-  idempotency_key UNIQUE constraint on payments table
 -  Audit log for all lease state transitions (append-only table)
 -  HTTPS enforced in production (nginx terminates TLS)
 -  Secrets via env vars only — never hardcoded
 -  CORS: explicit allow-list, no * in production
-

Questions before I start coding

1. Notifications — Twilio or Africa's Talking? The .env.example has Twilio credentials. For Uganda (MTN/Airtel subscribers), Africa's Talking has better delivery rates and lower cost. Should I build adapters for both and let config decide, or stick with Twilio only for now?

2. Payment gateway integration depth? The frontend records payments manually (user selects MTN/Airtel and enters a transaction reference). Should I:

- **(a)** Just persist what the user submits (simple, works now), or
- **(b)** Add webhook receivers for MTN MoMo API + Airtel Money API so payments auto-confirm?

Option (b) is the right long-term answer but adds 2–3 weeks of work. I'd suggest designing the schema for (b) but implementing (a) now.

3. Celery worker — same container or separate? For local dev I'll add a worker service to docker-compose.local.yml. In production, do you want it as a separate service (recommended — scales independently) or bundled with the API container?

4. PDF templates — any existing brand design to follow? For lease agreements I'll use Jinja2 + WeasyPrint. Should the PDF match the Crib brand (dark indigo, same fonts as the frontend), or is a clean legal-document style preferred?

5. Logto OIDC — is the Logto instance running locally or will I stub auth for now? I'll implement real JWKS validation either way, but if Logto isn't running yet I'll add a DEV_BYPASS_JWT=true env flag that accepts any bearer token and returns a mock user — similar to how the frontend has NEXT_PUBLIC MOCK_API=true.

Ready to start coding the moment you confirm (or override) these 5 questions. I'll begin with Sprint 1 (scaffolding) since that's independent of the answers, and can begin that now while you respond if you'd like.